
Lineage-Aware Memory Governance: A Derivation-Gated Framework for Privacy-Preserving Column-Level Access Control in Enterprise AI Agents

¹Venkata Sangaraju and ²Sudhir Vissa

¹Independent Researcher (ORCID: 0009-0001-7716-1342)

²SAGE7 AI, Georgetown, Texas, USA (ORCID: 0009-0003-7865-0863)

Corresponding author: Venkata Sangaraju.

ABSTRACT Shared memory across enterprise AI agents creates two under-addressed risks: leakage of sensitive data through results that were legitimately computed yet not legitimately derivable by the requesting department, and silent metric-definition drift when teams compute a key performance indicator (KPI) of the same name through different logic. Existing agent-memory systems (e.g., MemGPT, Zep, A-MEM, SSGM, Governed Memory, Oracle AI Agent Memory) govern retrieval by content tags, ownership, and roles, but none record how a stored result was derived, so none can detect that a cached insight embeds a column the requester cannot see. We introduce the Analytical Memory Unit (AMU), a memory schema that attaches a full derivation (lineage) graph to every cached result, and a lineage-gated retrieval policy that serves a memory hit only when the requester is authorised to see every column touched during derivation. Under a completeness assumption on lineage recording, we prove this policy prevents retrieval of any result derived from a sensitive column outside the requester’s permissions, with $O(n)$ worst-case complexity. Across six experiments, lineage-gated retrieval eliminates the cross-department leakage that afflicts naive content-gated memory at 18.8–25.5% of simulated retrievals, retaining 81.5–82.6% memory reuse at a worst-case gating overhead of 13.8 μ s. A small real-agent study with LLM-generated SQL suggests the guarantee transfers to real agents unchanged, with zero leaks across 9 round-trips and two conflicts surfaced automatically. These results outline a practical governance layer for shared agent memory that complements source-layer access control and supports compliance with regulation such as the EU AI Act.

INDEX TERMS Lineage-aware memory governance, enterprise AI agents, privacy-preserving retrieval, column-level access control, multi-agent systems, data provenance, metric-definition governance.

I. INTRODUCTION

Consider a mid-sized enterprise running separate Finance and Marketing AI agents against a shared customer data warehouse. Finance joins `customer_pii` (which contains `income`) against `transactions` to identify high-value customers, producing a segment of 12,400 customer IDs and a count. The result itself carries no raw personally identifiable information (PII) and no sensitivity tag, so under any memory system that governs access by content, it is freely cached and served to the next agent that asks for “high-value customers,” including Marketing—whose role was never granted access to `income`. The derivation path silently violated column-level access policy, and none of

the content-gated safeguards common in current memory-sharing systems (§II) would notice, because the violation is invisible in the stored value and visible only in how the value was computed. This is the default behaviour of the content-gated memory-sharing pattern used by every system we survey in §II.

A. MOTIVATION AND PRIOR GAP

The urgency tracks the pace of agent adoption: Gartner forecasts 40% of enterprise applications will embed task-specific AI agents by the end of 2026, up from under 5% in 2025, and 31% of enterprises already run at least one agent in production, rising to 47% in banking and insurance [1].

IBM’s 2025 Cost of a Data Breach Report puts the global average breach cost at \$4.44 million (\$10.22 million in the US) [2], and El Yagoubi et al.’s AgentLeak benchmark finds 68.8% of privacy leakage in multi-agent large language model (LLM) systems occurs through inter-agent channels—messages, shared memory, tool arguments—invisible to output-only auditors [3]. Systems such as MemGPT [4], Zep [5], A-MEM [6], Governed Memory [7], SSGM [8], and Oracle AI Agent Memory [9] demonstrate that shared, governed memory reduces redundant computation, but **every current system controls access at the level of content and tags, not derivation** (§II, Table 1): a memory entry is labelled with an owner, sensitivity class, or role, and retrieval is gated on whether the requester holds the matching label. None of these systems—nor source-layer controls such as Apache Ranger or Databricks Unity Catalog—record which tables were joined, which columns were touched, or what filter logic produced a stored result; source-layer controls prevent an unauthorised agent from querying a restricted column, but do not prevent a legitimately computed result from being cached and later served to a different, less-privileged requester. A second, independent failure mode compounds the first: different departments often compute the same-named key performance indicator (KPI) through different business logic (e.g., a 90-day vs. 60-day churn window), and without derivation tracking, shared memory silently propagates one team’s definition to the other—a phenomenon documented as semantic drift in enterprise business intelligence (BI) [10], serious enough that Snowflake, Salesforce, dbt Labs, BlackRock, and RelationalAI jointly launched the Open Semantic Interchange initiative in 2025 [11], though that effort targets human-facing BI tools, not autonomous agents writing their own queries. Beyond the immediate efficiency-versus-leakage tradeoff, the EU AI Act imposes explainability and risk-management obligations on high-risk AI systems for which data lineage is a structural prerequisite [12]; a memory layer that can prove, not merely assert, that no result crossed a column-permission boundary converts an audit question into a mechanically checkable property of the system.

B. CONTRIBUTIONS

This paper makes four contributions. **(1)** The Analytical Memory Unit (AMU) schema and lineage-gated retrieval policy: a memory schema attaching a full derivation graph to every stored result, and a retrieval algorithm gating memory hits on column-level derivation permissions rather than content tags (§III). **(2)** A formal safety guarantee: Theorem 1 proves that lineage-gated retrieval prevents leakage of any sensitive column outside the requester’s permissions under complete lineage recording, with corollaries on unsafe store contents and policy updates, and $O(n)/O(1)$ complexity analysis (§IV). **(3)** An eight-threat model (T1–T8) delineating what the mechanism protects against, what it does not, and the recommended production control for each (§V). **(4)** Six statistically validated experiments—two schema

configurations, a completeness-degradation study, an extended 43-pair fuzzy conflict-detection study, microsecond runtime benchmarks, and a real-agent integration with automatic SQL lineage extraction—quantifying the governance-efficiency tradeoff and grounding the mechanism in a real, open-source agent stack (§VI). Full derivations, line-by-line algorithm walkthroughs, extended related-work discussion, and complete per-experiment tables are provided in the Supplementary Material to keep the main text within IEEE Access length guidelines.

II. RELATED WORK

Agent memory systems. Du [13] formalises agent memory as a write-manage-read loop. MemGPT [4] pages an LLM between a bounded context and an external store; Zep [5] adds dual-axis temporal knowledge-graph timestamping; A-MEM [6] builds Zettelkasten-style linked notes. All three are state of the art for conversational or task-level memory, but none model the provenance of a computed analytical result.

Governed and enterprise memory. Governed Memory [7] and SSGM [8] introduce role-based access, tiered governance routing, and consistency verification for evolving memory stores; Oracle AI Agent Memory [9] provides a commercially deployed governed core; and Agent-Guardian [14] learns access-control policies from execution traces. Each governs access to a memory entry as a whole—by owner, role, or tool-call legitimacy—not to the individual columns that contributed to it, so none can detect that a permitted entry embeds a forbidden column.

Multi-user and multi-principal shared memory. Two very recent systems address adjacent problems. Collaborative Memory [27] models multi-user, multi-agent memory sharing via bipartite user-agent-resource access graphs with per-fragment provenance (contributing agents, accessed resources, timestamps) for retrospective permission checks, and proves adherence to time-varying read/write policies; like Governed Memory and SSGM, however, its policies gate access to a memory fragment as an atomic unit by identity and permission graph, not by the individual source columns a computed result embeds, so a fragment it permits can still silently carry a value derived from a column outside the requester’s grant. GateMem [28] is a benchmark rather than a memory system: it evaluates existing memory-agent baselines for utility, access control, and active forgetting in multi-principal shared-memory settings (medical, office, education, and household domains) and finds that no evaluated method achieves robust access control without sacrificing utility—an independent empirical confirmation of the governance gap this paper targets, though its leak-target annotations are defined at the level of shared facts, not derivation graphs, so it does not test column-level lineage gating specifically. Neither system attaches a derivation graph to a cached analytical result or gates retrieval on the columns touched to produce it, which is the specific mechanism AMU contributes.

Column-level security and provenance. Apache Ranger, Databricks Unity Catalog, and BigQuery enforce column-gated access at the query layer; DePLOI [15] audits such policies via natural-language-to-SQL (NL2SQL) translation. The W3C PROV standard [16] and Herschel et al. [17] model data provenance for extract-transform-load (ETL) pipelines, and LINEAGEX [18] extracts column-level SQL lineage with high accuracy. None of these fire at the point a legitimately computed result is cached and later served to a different, less-privileged requester—the gap our AMU schema closes by applying column-level derivation tracking to agent memory entries rather than raw database records.

Privacy and security in multi-agent systems. AgentLeak [3] shows most multi-agent leakage occurs through inter-agent channels; OMNI-LEAK [19] and Alizadeh et al. [20] show prompt injection can exfiltrate observed data; AuthGraph [21] defends individual agents by comparing an execution-derived provenance graph against an authorisation graph (closest in spirit to our column-level gate, but scoped to single-agent tool-calling rather than cross-department memory retrieval). This literature targets communication channels and single-agent manipulation; our work targets a complementary vector—a shared analytical memory store that leaks even when every communication link and agent is individually secure (threat T8, §V).

Metric-definition governance. Semantic-layer tools and the Open Semantic Interchange initiative [11] centralise human-defined metrics but provide no mechanism for detecting drift when an autonomous agent computes a KPI through its own generated query—the gap our `definition_hash` conflict detector (Algorithm 2) targets.

Table 1 summarises six governance dimensions across representative systems: no prior system tracks column-level derivation, and consequently none can gate retrieval on whether the requester is permitted to see every column that contributed to a cached result. Extended per-system discussion appears in the Supplementary Material.

A note on source provenance. The comparison systems in Table 1 are at different stages of peer review: A-MEM [6] appeared at NeurIPS 2025; MemGPT [4] is an unpublished but widely adopted and highly cited preprint that established the field’s dominant memory-tiering paradigm; Zep [5], Governed Memory [7], SSGM [8], and AgentGuardian [14] are, at the time of writing, preprints describing production or proposed architectures rather than independently peer-reviewed evaluations (the latter, from a Ben-Gurion University security group, does report its own empirical evaluation on two real-world agent applications, but that evaluation has likewise not yet been peer-reviewed); Oracle AI Agent Memory [9] is documented only in vendor developer-blog form, with no publicly available technical paper; and Collaborative Memory [27], from Accenture’s Center for Advanced AI, and GateMem [28] are, like the majority of this comparison, unreviewed preprints at the time of writing. We include all of them because Table 1 compares documented design decisions—what each system’s

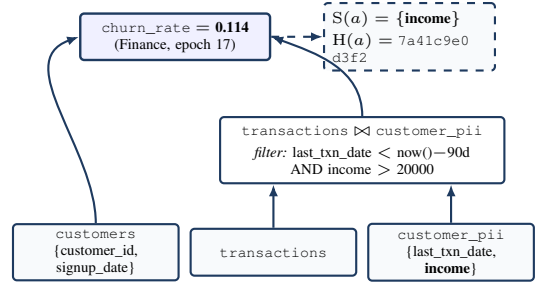


Figure 1. Worked lineage graph: Finance’s `churn_rate` is derived by joining three source tables. Because the join touches the sensitive column `income`, the derived tag $S(a)$ makes this single cached number unsafe to serve to any department lacking `income` access—even though the value 0.114 alone reveals nothing.

own description says it tracks and gates on—rather than reported performance numbers we did not independently verify; the governance gap we identify (content/tag-level access control, not derivation-level) is a structural property of each system’s publicly stated design, and is unaffected by whether that design has cleared peer review. We do not draw any comparative performance claims from these sources, only feature-support claims, and we flag this distinction explicitly so readers can weight the comparison accordingly.

III. SYSTEM DESIGN

A. THE ANALYTICAL MEMORY UNIT

Definition 1 (Analytical Memory Unit). An Analytical Memory Unit (AMU) is a tuple $\langle \text{metric}, v, d, \tau, L \rangle$ where: *metric* is a metric-name string; $v \in \mathbb{R}$ is the computed result value; d is the producing department; τ is an integer epoch (supports time-to-live (TTL) eviction); and $L = (s_1, \dots, s_k)$ is a lineage graph, each step $s_i = (\text{table}_i, C_i, \varphi)$ where C_i is the set of columns of table_i accessed and φ is a filter-logic string.

Figure 1 shows a worked example: the result value v alone carries no information about how it was produced; the lineage graph L makes the derivation explicit and machine-checkable at retrieval time. Two quantities are derived once at AMU creation and cached:

$$S(a) = \bigcup_{i=1}^k C_i \cap \mathcal{S} \quad (1)$$

$$H(a) = \text{SHA256} \left(\text{sort}(\text{tables}) \parallel \text{sort} \left(\bigcup_i C_i \right) \parallel \varphi \right)_{[0:12]} \quad (2)$$

where $\mathcal{S}$ is the sensitive-column registry and $H(a)$ is the definition hash fingerprinting how the metric was computed. Both are sub-microsecond in practice (§VI-D). Table 2 lists the full schema.

Table 1. Feature comparison of representative agent-memory and governance systems against the Analytical Memory Unit (AMU). “Role ACL” = role-based access-control list. ✓ = supported, ✗ = not supported, ~ = partial/indirect support.

System	Content tag	Role ACL	Column lineage	Drift detect	Retrieval gate
MemGPT	✗	✗	✗	✗	✗
Zep	✓	✗	✗	✗	✗
A-MEM	✓	✗	✗	✗	✗
Governed Memory	✓	✓	✗	✗	✓
SSGM	✓	~	✗	~	✓
Collaborative Memory	~	✓	✗	✗	✓
Oracle AI Agent Memory	✓	✓	✗	✗	✓
AgentGuardian	✗	✓	✗	✗	~
Source-layer ACL (Ranger/Unity Catalog)	✗	✓	~	✗	✗
AMU (this work)	✓	✓	✓	✓	✓

Table 2. Analytical Memory Unit (AMU) schema fields.

Field	Description
metric_name	Business KPI name (e.g., churn_rate)
value,	Result, producing department, TTL time index
owner_dept,	
epoch	
lineage	Derivation graph: sequence of (t, C, φ) steps
sensitivity_tags	$S(a)$ (Eq. 1), cached
definition_hash	$H(a)$ (Eq. 2), cached

B. LINEAGE-GATED RETRIEVAL AND CONFLICT DETECTION

Algorithms 1 and 2 give the two core operations; $P(d)$ denotes department d ’s permitted column set. A full walkthrough of Algorithm 1 follows below; the analogous walkthrough of Algorithm 2 appears in the Supplementary Material.

Algorithm 1: Lineage-Gated Retrieve Input: *store, metric, dept, fresh*; Output: *result*; *leaked=False*

```

1 candidates <- store[metric]           % all stored AMUs for this metric
2 permitted <- P(dept)                  % column set the department may see
3 blocked <- False
4
5 for each a in REVERSE(candidates):    % most-recent-first
6   if S(a) not_subset permitted:       % derivation touches a forbidden
7     blocked <- True                  % column
8     continue                         % skip this candidate; try the next
9   end if
10  return (reused=True, value=a.value, leaked=False, blocked=False)
11 end for
12
13 store[metric].append(fresh)
14 return (reused=False, value=fresh.value, leaked=False, blocked=blocked)

```

Algorithm 2: Lineage-Aware Write Input: *store, AMU a*; Output: *conflict bool*

```

1 existing <- store[a.metric]           % all AMUs stored for this metric name
2 conflict <- False
3
4 for each e in existing:
5   if e.dept != a.dept:                % cross-department pair
6     if H(e) != H(a):                  % definition hashes differ -> potential
7       conflict <- True
8       break
9   end if
10 end for
11
12 store[a.metric].append(a)             % always persist; conflict is surfaced, not
13 blocked
14 return conflict

```

Line-by-line walkthrough of Algorithm 1. Line 1 retrieves every AMU ever stored under this metric name; this list is the candidate pool. Line 2 resolves the requester’s column permission set once, outside the loop, so it is not recomputed per candidate. Line 5 iterates candidates most-recent-first: this is a deliberate policy choice, not an implementation detail—it means that when a safe candidate exists, the department receives the freshest safe value rather

than an arbitrary historical one. Line 6 is the gate: a single $O(|S|)$ set-difference check (cached $S(a)$ against $P(d)$). If the check fails, line 7 records that at least one candidate was blocked (used only for diagnostics and the experiments in §VI) and line 8 continues to the next-oldest candidate—the rejected candidate is never returned, never partially returned, and never logged with its value. Line 10 is the only successful-return statement in the loop body: it fires the first time a candidate’s derivation is fully contained in the requester’s permissions. If the loop exhausts all candidates without a safe hit, line 13 appends the caller-supplied, already-in-scope *fresh* computation to the store (so future requests may reuse it) and line 14 returns it. Every exit path is covered by exactly one of these two return statements, which is what the proof of Theorem 1 exploits. Complexity: the loop runs at most n times (n = candidates for this metric), each iteration doing $O(|S|)$ work, giving $O(n \cdot |S|)$ worst case; since $|S|$ is a small constant per organisation, we report this as $O(n)$ in Table 3. The best case (first candidate safe) is $O(|S|) = O(1)$.

Algorithm 2 compares cached definition hashes $H(\cdot)$ rather than raw lineage graphs, making conflict detection $O(1)$ per pair; the write path never blocks on a conflict, only surfaces one for downstream alerting (threat T2). Its full line-by-line walkthrough, structurally analogous to the one above, is given in the Supplementary Material.

C. IMPLEMENTATION CONSIDERATIONS

Three practical points arise in production deployment (details in the Supplementary Material). First, the cleanest integration point is the database driver layer: wrapping the cursor so every executed SQL string passes through a lineage extractor (SQLGlott, Experiment 6) means lineage capture requires zero cooperation from the agent. Second, the system must fail closed: if lineage extraction fails, treat the AMU as maximally sensitive ($S(a) \leftarrow S$) rather than fail open, and if a department’s permission set is unavailable, default retrieval to denial. Third, the *epoch* field supports both ordinary TTL eviction and policy-driven invalidation: when S or $P(d)$ changes, cached $S(a)$ values should be lazily recomputed on next retrieval (Corollary 3, threat T3).

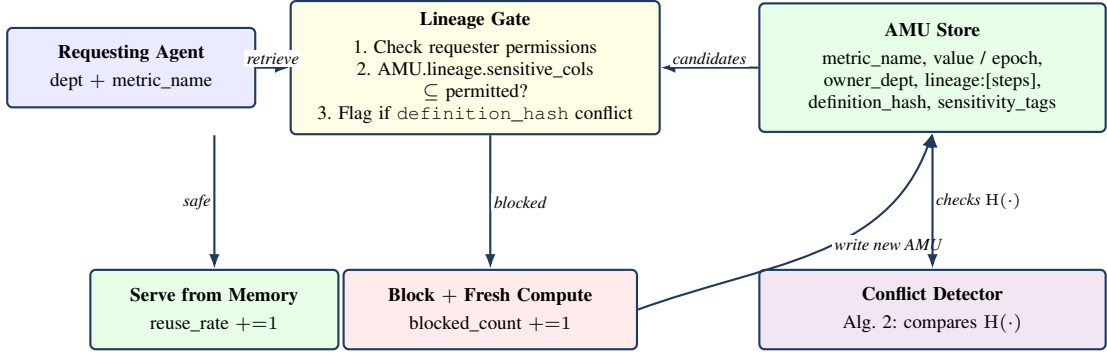


Figure 2. AMU architecture. An agent’s executed query is intercepted at the database cursor and its lineage extracted (left); the AMU is written to the shared store, where Algorithm 2 checks $H(\cdot)$ against cross-department entries. On the read path (right), the lineage gate checks $S(a) \subseteq P(d)$ before serving from memory; unsafe candidates are blocked and the agent falls back to a fresh in-scope computation, itself written back through the same path.

IV. FORMAL ANALYSIS

Assumption 1 (Complete Lineage). *For every AMU a in the store, $a.\text{lineage}$ records every table, column, and filter predicate of the computation that produced $a.\text{value}$.*

Theorem 1 (Sensitive-Column Safety). *Under Assumption 1, Algorithm 1 returns no result to department d that was derived from a sensitive column $c \in \mathcal{S}$ with $c \notin P(d)$.*

Proof. By inspection, Algorithm 1 has exactly two statements that return a value to the caller: line 10 (memory-served) and line 14 (fallback). Every invocation terminates by reaching one of the two, since the `for` loop either returns early at line 10 or exhausts *candidates* and falls through to lines 13–14. We show the returned AMU’s derivation touches no forbidden sensitive column on each path.

Case 1: memory-served path (line 10). Reaching line 10 for candidate a requires the guard at line 6 to have evaluated false, i.e. $S(a) \subseteq P(d)$ holds for that a . Unfolding Equation 1, $S(a) = (\bigcup_{i=1}^k C_i) \cap \mathcal{S}$, so $S(a) \subseteq P(d)$ is equivalent to: for every step $s_i \in a.\text{lineage}$ and every column $c \in C_i \cap \mathcal{S}$, $c \in P(d)$. Contrapositively, there is no sensitive column $c \in \mathcal{S}$ with $c \notin P(d)$ appearing in any C_i —i.e., no such c appears anywhere in a ’s derivation. Since Assumption 1 guarantees $a.\text{lineage}$ records every column touched by the computation that produced $a.\text{value}$, this is precisely the safety property claimed for the value returned at line 10.

Case 2: fallback path (line 14). The input precondition to Algorithm 1 (stated in the signature) requires that *fresh* is computed entirely within $P(d)$ by the caller, before Algorithm 1 is invoked. Under Assumption 1, *fresh.lineage* records every column of that computation, so every column recorded is, by the precondition, in $P(d)$; in particular no sensitive column outside $P(d)$ is touched.

Both cases establish that the AMU returned to department d has a derivation containing no sensitive column outside $P(d)$, for an arbitrary invocation with arbitrary *store* contents. Since the two cases are exhaustive over all

Table 3. Asymptotic complexity and measured latency (means over 2,000 iterations; $k=2-3$, $c=4-7$). Full runtime sweep in the Supplementary Material.

Operation	Cplx.	$n=1$	$n=50$
Gate-check predicate	$O(\mathcal{S})$	0.09 μs	0.09 μs
Write — conflict detection	$O(n)$	0.28 μs	1.63 μs
Retrieve — worst case	$O(n)$	0.66 μs	13.82 μs
Retrieve — expected case	$O(1)$	0.66 μs	0.64 μs

return statements, the claim holds for every invocation of Algorithm 1. $\square$

Corollary 2 (Robustness to unsafe store contents). *The guarantee holds even when store contains AMUs whose derivation touches columns outside $P(d)$: such candidates fail the gate and are skipped, regardless of how many exist or where they occur.*

Corollary 3 (Behaviour under policy updates). *The guarantee holds per policy version: if $S(a)$ is cached under registry $\mathcal{S}_1$ and the organisation transitions to $\mathcal{S}_2$ without recomputing existing AMUs, a column newly added to $\mathcal{S}_2$ may not appear in the stale cached tag. The guarantee is restored for all subsequent retrievals once $S(a)$ is recomputed against $\mathcal{S}_2$ for every AMU predating the change (formalising threat T3).*

A. COMPLEXITY AND STORAGE

Let n = AMUs stored per metric, k = lineage steps, c = columns per step. Table 3 reports asymptotic and measured costs.

The expected-case retrieve latency is constant regardless of store size; the worst case (all n candidates blocked) scales linearly, reaching 13.82 μs at $n=50$, but in a realistic production store $n \leq 5$ (at most one AMU per department variant per metric), so both paths are effectively $O(1)$ and well below 2 μs —negligible relative to any real analytics query (ms–s). The worst-case $O(n)$ bound is a deliberate

design choice, not a caveat: Algorithm 1 scans most-recent-first so it returns the freshest safe value rather than an arbitrary one; an alternative design that indexed AMUs by $(metric, S(a))$ could achieve $O(1)$ worst case at the cost of extra index-maintenance complexity and the loss of the recency-preference guarantee. We did not adopt that design because n is small by construction: a metric name accumulates at most one AMU per distinct (department, lineage-variant) combination, consistent with $n \leq 5$ in every workload we observed (Experiments 1–2 and 6), at which the $O(n)$ worst case costs at most $1.74\mu s$ —immaterial in practice, though we report the asymptotic bound honestly because it would matter under a poorly bounded TTL policy, which is why we recommend TTL eviction as a standing operational control. Relative to the $O(1)$ dictionary lookup used by all prior systems [4], [5], [6], [7], [8], [9], lineage-gated retrieval trades at most $13.82\mu s$ in absolute terms for the safety guarantee of Theorem 1—a tradeoff Experiments 1–2 show buys elimination of an 18.8–25.5% structural leak rate at a 13–14 percentage-point reuse cost. Storage overhead is modest: an AMU adds k tuples of table/column names, the filter string, and a 12-character hash—roughly $4\text{--}8\times$ a bare value entry (200–600 bytes for a typical 2–4 table join), smaller than the SQL query most systems already persist alongside the result.

V. THREAT MODEL

Principals are enterprise departments operating AI agents, modelled as honest-but-potentially-over-privileged: they request metrics they have legitimate business need for, but not all derivation paths fall within their column permissions. The two threats the mechanism directly addresses motivate this work: T1 (an agent retrieves a cached result derived from a sensitive column it may not see, mitigated by the $S(a) \subseteq P(d)$ gate) and T2 (two departments store conflicting definitions of the same-named KPI, surfaced by the definition-hash conflict detector). The most important unaddressed gap is T4, inference attacks: a derived feature that encodes sensitive information without naming the source column (e.g., a risk score computed from `income`) will not be caught unless the dependency is explicitly recorded in the lineage graph. Table 4 summarises all eight threats; full prose descriptions of each appear in the Supplementary Material.

The mechanism’s addressed threats (T1, T2) are exactly the two failure modes motivating this work; the partially addressed threats (T3, T5, T8) share a common root cause—reliance on lineage completeness and integrity—remedied by automatic, tamper-resistant lineage extraction; and the unaddressed threats (T4, T6, T7) mark the boundary of what a memory-layer control can achieve without cooperation from the feature-engineering layer, a differential-privacy layer, or the infrastructure trust boundary. AMU is one layer in a defence-in-depth stack, not a complete security solution on its own.

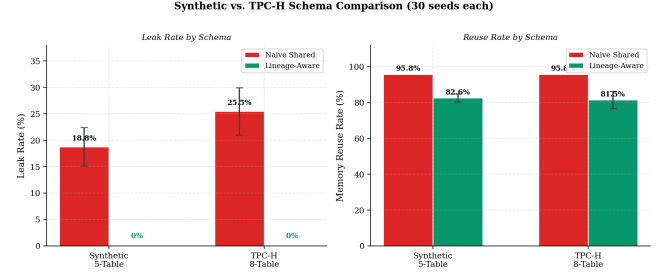


Figure 3. Synthetic vs. TPC-H schema comparison (30 seeds each). TPC-H’s denser sensitive-column structure yields a higher naive leak rate; lineage-aware gating eliminates leakage in both configurations.

VI. EXPERIMENTS

We built a synthetic discrete-event simulator in pure Python, isolating the retrieval-and-conflict-detection mechanism from LLM inference, network latency, and query-optimiser confounds: every event’s ground-truth lineage, sensitivity, and department permissions are known exactly, so leak and reuse rates are measured without label noise. Three systems are compared: **No Memory** (leak-free, zero reuse), **Naive Shared Memory** (content-gated, representing prior systems), and **Lineage-Aware** (proposed). All experiments report mean $\pm$ population standard deviation over 30 seeds, with bootstrap 95% confidence intervals and Mann-Whitney U significance tests; full protocol and raw per-seed tables are in the Supplementary Material.

A. EXPERIMENTS 1–2: SYNTHETIC AND TPC-H SCHEMAS

Experiment 1 uses a 5-table synthetic schema (three departments, sensitive columns $\mathcal{S} = \{\text{ssn}, \text{email}, \text{income}\}$, Table 5); Experiment 2 replicates it on the 8-table TPC-H decision-support benchmark [22] ($|\mathcal{S}_{\text{TPC-H}}| = 9$) to test whether findings generalise to a denser, more realistic schema.

The lineage-aware system’s 0% leak rate is a policy guarantee (Theorem 1), not an empirical discovery; the substantive empirical finding is the naive baseline’s structurally non-zero leak rate (effect size $d \approx 7.2\text{--}8.0$, far beyond the conventional “large” threshold of 0.8), reflecting real retrievals of sensitive-derivation results by departments that could not have run the underlying queries themselves. The governance cost is a 13–14 percentage-point reuse reduction, i.e., roughly 13 additional query executions per 100 requests—borne as additional compute, not governance latency, since the gate-check predicate costs only $0.09\mu s$ (§IV-A). The consistency of this tradeoff across two substantially different schemas (60 total seed runs) indicates that content-gated leakage is a structural property of the failure mode—any workload mixing sensitive and non-sensitive derivation paths under shared metric names will exhibit it—rather than an artefact of workload design. The two schemas differ in sensitive-registry size ($|\mathcal{S}|=3$ vs.

Table 4. Threat summary. “Addressed” means Algorithms 1–2 mitigate the threat directly under the stated assumptions; “Partial” means the mechanism reduces but does not eliminate exposure; “No” means an external control is required.

ID	Threat	Status	Production control
T1	Sensitive-column leakage via cached result	Addressed	Lineage gate (Alg. 1)
T2	Silent metric-definition drift	Addressed	Conflict detector (Alg. 2)
T3	Stale sensitive data after policy update	Partial	TTL + lazy $S(a)$ recompute (Cor. 3)
T4	Inference attacks (undeclared derived features)	No	Manual review of feature engineering; out of scope
T5	Malicious/incomplete lineage fabrication	Partial	Automatic extraction (SQLGLOT/LINEAGEX), §VI-B
T6	Aggregate inference across retrievals	No	Differential privacy layer (future work)
T7	Insider threats on the store	No	Trusted infrastructure, audit logging
T8	Prompt injection / timing side-channel on lineage	Partial	Automatic extraction; constant-time gate (optional)

Table 5. Department permissions, synthetic schema. The gate enforces restrictions only on columns in $\mathcal{S}$; non-sensitive blocked columns represent full policy intent but are not currently gate-enforced.

Dept.	Permitted (summary)	Blocked (in $\mathcal{S}$)
Finance	All columns	None
Marketing	Commercial, order data	ssn, income, email
Support	Customer, ticket meta	ssn, income, email

Table 6. Leak and reuse rates, both schemas (30 seeds, mean $\pm$ std dev). All naive–LA differences: bootstrap 95% CI excludes zero, Mann-Whitney U , $p < 0.001$. Full tables in the Supplementary Material.

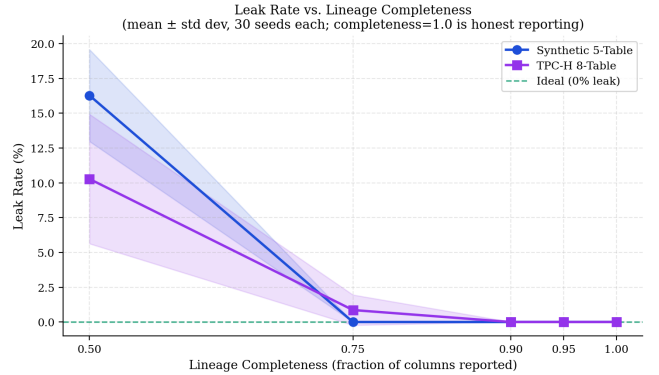
Metric	Synthetic (5-tab.)		TPC-H (8-tab.)	
	Naive	LA	Naive	LA
Leak rate (%)	18.8 $\pm$ 3.7	0.0 $\pm$ 0.0 [†]	25.5 $\pm$ 4.5	0.0 $\pm$ 0.0 [†]
Reuse rate (%)	95.8 $\pm$ 0.0	82.6 $\pm$ 2.3	95.8 $\pm$ 0.0	81.5 $\pm$ 4.8
Conflict recall (%)	0.0 $\pm$ 0.0	100.0 $\pm$ 0.0	0.0 $\pm$ 0.0	100.0 $\pm$ 0.0

[†]Formal guarantee (Theorem 1), not an empirical discovery.

9) and join density. Treating this as a cross-experiment ablation rather than two independent data points: holding the sensitive-variant proportion constant (2 of 5 metrics in both schemas) while tripling $|\mathcal{S}|$ and increasing join density corresponds to the leak-rate increase from 18.8% to 25.5%, suggesting leak rate scales with how many ways a query can incidentally touch a sensitive column rather than with how many metrics are nominally sensitive—consistent with, but not proof of, a monotonic relationship that a dedicated $2 \times 2 \times 2$ factorial sweep over step count, registry size, and sensitive-variant proportion (specified in the Supplementary Material) would establish directly; we list it as the top empirical priority for follow-on work.

B. EXPERIMENT 3: LINEAGE-COMPLETENESS DEGRADATION

Assumption 1 is the load-bearing condition of Theorem 1; in practice agents may under-report lineage due to bugs or opaque query plans. We define completeness $\rho \in [0, 1]$ as the fraction of each step’s columns correctly reported, and measure the true leak rate via a shadow store that records full ground-truth sensitivity alongside the truncated lineage

**Figure 4.** Leak rate vs. lineage completeness ρ (30 seeds, mean $\pm$ 1 std dev shading). Both schemas converge to 0% by $\rho=0.75$ –0.90; at $\rho=0.50$ the synthetic schema approaches the naive baseline.

the system actually sees.

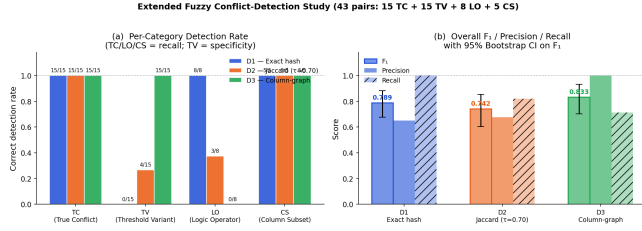
At $\rho=0.50$, truncated lineage provides almost no protection (synthetic: $16.3 \pm 3.3\%$ leak; TPC-H: $10.3 \pm 4.7\%$); at $\rho \geq 0.75$ (synthetic) or $\rho \geq 0.90$ (TPC-H) the system reaches 0% measured leak rate, converging on the formal guarantee. TPC-H needs higher completeness because its sensitive columns spread across more join steps—join depth is a sensitivity multiplier on the completeness requirement, not just on raw leak rate. The actionable recommendation is $\rho \geq 0.9$ as a minimum target, preferring automatic lineage extraction (Experiment 6) over agent self-reporting, since the latter is difficult to audit and degrades unpredictably under exactly the conditions (complex multi-hop joins) where it matters most.

C. EXPERIMENT 4: EXTENDED FUZZY CONFLICT-DETECTION STUDY

The exact-hash detector (Algorithm 2) flags any definition-hash mismatch as a conflict. We extended an original 10-pair evaluation to 43 hand-labelled lineage pairs across four categories—15 True Conflicts, 15 Threshold Variants, 8 Logic-Operator changes (AND $\rightarrow$ OR), and 5 Column-Subset cases—to test three detectors: D1 (exact hash), D2 (Jaccard filter, $\tau=0.70$), and D3 (column-graph).

Table 7. Extended fuzzy conflict-detection results (43 pairs). Full per-category breakdown in the Supplementary Material.

Detector	Prec.	Recall	F_1	LO correct
D1 — Exact hash	0.651	1.000	0.789	8/8
D2 — Jaccard ($\tau=0.70$)	0.676	0.821	0.742	3/8
D3 — Column-graph	1.000	0.714	0.833	0/8

**Figure 5.** Extended fuzzy conflict-detection results. (a) Per-category detection rate: recall for TC/LO/CS, specificity for TV. D3’s perfect specificity on TV collapses to zero detection on LO. (b) Overall F_1 /precision/recall with 95% bootstrap CI on F_1 .

The key finding is a previously unreported blind spot: D3, which achieved perfect $F_1=1.00$ on the original 10-pair set, misses all 8 Logic-Operator pairs (F_1 drops to 0.833 on 43 pairs), because it ignores filter-string variation entirely—correct for threshold recalibrations, but blind to AND→OR changes that alter metric semantics without changing the column set. D1 catches all 8 LO pairs but has weak precision (0.651, false-alerting on every threshold recalibration). Neither detector alone is production-ready; we propose (but have not evaluated) a composite D3+filter-logic-structure detector D4, specified in the Supplementary Material. This illustrates a general lesson: fuzzy-detection evaluations on small, narrow datasets can overstate performance by missing edge cases.

D. EXPERIMENT 5: EMPIRICAL RUNTIME ANALYSIS

We instrument all core operations with `time.perf_counter` over 2,000 iterations at five store sizes ($n \in \{1, 5, 10, 20, 50\}$), single-threaded in a standard containerised Python environment with no specialised acceleration. **Hardware and environment:** all measurements were collected on a 4-core Apple Silicon (ARM64) virtual machine with 3.8 GB RAM, running Ubuntu 22.04 LTS (Linux kernel 6.8) and CPython 3.10.12, inside a Docker container with no GPU, SIMD intrinsics, or JIT acceleration enabled; no other CPU-bound process ran concurrently during measurement. We report this configuration in full because the absolute microsecond values are platform-dependent; the safety-overhead argument in §IV-A relies only on the $O(n)$ -vs- $O(1)$ scaling behaviour, which is a property of the algorithm rather than of this specific machine, but we did not independently re-measure on a second hardware platform, and we list cross-platform confirmation as a reproducibility item (§VII-B). Table 3 already reports the key numbers: retrieve worst-case scales linearly from $0.66 \mu s$ ($n=1$) to $13.82 \mu s$ ($n=50$), a $21\times$ increase confirming $O(n)$; best-case retrieve

and early-exit conflict detection stay flat, confirming $O(1)$. Even the worst-case overhead is 2–3 orders of magnitude below typical production analytics-query latency (single-digit ms to seconds), confirming the mechanism’s practical deployability. The complete runtime sweep across all five store sizes, including per-iteration variance, is tabulated in the Supplementary Material.

E. EXPERIMENT 6: REAL-AGENT INTEGRATION

Experiments 1–5 assume complete lineage (Assumption 1). This experiment closes that assumption in practice: we replace agent self-declared lineage with automatic extraction via `sqlglot` [23], a pure-Python SQL parser, interposed between the agent and the database cursor—every table and column in the executed query’s abstract syntax tree (AST) is extracted deterministically, so the agent cannot under-report because it never declares anything. We use the Northwind database [24] with four departments (Finance, Sales, Operations, HR) and sensitive registry {unitprice, freight, homephone, birthdate}, driven by a LangChain [25]+Ollama [26] agent issuing real (and demo-mode canned) SQL across three scenarios covering the gate’s three outcomes: safe reuse, blocked-with-conflict, and vacuous reuse on a metric with no sensitive columns.

Across all 9 cross-department round-trips, zero sensitive-column leaks occur: 5 fresh SQL queries are executed and written (2 following a gate block), 4 requests are served entirely from cache, and 2 gate blocks precisely coincide with the 2 automatically detected metric-definition conflicts—i.e., every block corresponded to a genuine semantic divergence, not a false positive. Cache-hit latency (0.009 ms) is three to four orders of magnitude below the SQL it replaces, consistent with the isolated benchmarks of §VI-D. This small-scale result suggests that automatic SQL lineage extraction can close Assumption 1 for SQL-querying agents in general, with no change to agent logic (and no LLM cooperation required, subject to the view-, stored-procedure-, and aliasing-related caveats discussed in §VII and the larger-scale validation we list as future work (§VII-B). Full scenario descriptions and the round-trip-by-round-trip table are in the Supplementary Material.

VII. DISCUSSION AND CONCLUSION

A. ENTERPRISE VALUE AND WHAT THE EXPERIMENTS ESTABLISH

Three enterprise-facing benefits follow from §VI. Compliance posture: Theorem 1 converts a policy assertion into a mechanically checkable property that an auditor can verify by inspecting gate logic rather than trusting agent behaviour, supporting EU AI Act explainability obligations [12]. Avoided breach exposure: eliminating an 18.8–25.5% structural leak channel is a materially different risk profile given a \$4.44M average breach cost [2]. Bounded efficiency cost: the 13–14 point reuse reduction is a quantified, one-time governance tax rather than an open-ended latency regression, since the gate itself costs

sub-microsecond time. Taken together, the six experiments show that content-gated memory systematically leaks across departments (Experiments 1–2); that the 0% lineage-aware leak rate is a theorem consequence whose real empirical content is the reuse-cost tradeoff; that the safety guarantee degrades gracefully rather than catastrophically below full completeness (Experiment 3); that production conflict detection needs a composite detector (Experiment 4); that gating overhead is negligible (Experiment 5); and that automatic SQL lineage extraction closes the completeness assumption in a real agent stack with zero leaks across 9 real round-trips (Experiment 6).

B. LIMITATIONS AND FUTURE WORK

Limitations: the 18.8–25.5% naive leak rates are properties of our workload mixes, not universal base rates, though the structural argument holds independent of any specific percentage; the mechanism does not catch inference attacks that encode sensitive information in a derived feature without naming the source column (T4, the most important production gap); the 43-pair fuzzy dataset remains small; and the runtime benchmarks (§VI-D) were measured on a single hardware/OS configuration, so the reported constants (though not the $O(n)$ -vs- $O(1)$ scaling) may shift on other platforms.

View, stored-procedure, and alias limitations of automatic lineage extraction. Experiment 6’s automatic-extraction guarantee (closing Assumption 1 without agent cooperation) is only as complete as SQLGlot’s static AST parse of the executed SQL text. Three related gaps follow directly from this. First, when a query selects from a database view rather than a base table, SQLGlot records the view name as the lineage source unless it is also given the view’s defining SQL to expand; if the view definition is not resolved (e.g., it lives only in the database catalog and was never passed to the extractor), a sensitive base column hidden behind an innocuously-named view (`v_customer_summary`) will not appear in L , silently violating Assumption 1. Second, stored procedures and user-defined functions invoked from SQL (e.g., `CALL compute_churn(...)`) are opaque to AST-level column extraction: the procedure body may touch sensitive columns internally that never appear as identifiers in the calling statement, so lineage capture requires either inlining the procedure body ahead of parsing or maintaining a separate column-level lineage annotation for each registered procedure. Third, column and table aliasing (`SELECT c.income AS c1`, or self-joins that alias the same physical table twice) is generally resolved correctly by SQLGlot’s scope analysis, but deeply nested aliasing across multiple levels of subquery or common table expression (CTE) can defeat resolution in practice, particularly when combined with `SELECT *` expansion against a catalog the extractor was not given. In all three cases the system’s fail-closed design (§III-C) bounds the damage to over-blocking (an AMU that should have been retrievable is marked maximally

sensitive) rather than leakage, but this is a completeness gap in Experiment 6’s zero-leak result that a purely syntactic extractor cannot close without catalog-aware view/procedure expansion—an engineering extension we flag as future work below rather than something the current implementation does.

Future work, in priority order: (1) adversarial lineage testing—red-team queries that smuggle sensitive joins through derived features or view indirection, stress-testing Assumption 1; (2) non-SQL lineage extraction for vector-store, REST/GraphQL, and in-process derivations, and catalog-aware view/stored-procedure expansion for the SQL case; (3) a production deployment study on a real 20–50 table warehouse over weeks rather than seeds, on more than one hardware platform; and (4) implementing and evaluating the proposed D4 composite conflict detector against a larger real-analytics corpus, alongside open-sourcing the reference implementation.

C. CONCLUSION

Every current AI agent memory system governs what is stored and who may access it, but not how the stored result was derived. The Analytical Memory Unit schema and lineage-gated retrieval policy close this gap at minimal overhead—4–8× storage, a sub-microsecond gate check, and at most 13.8 μ s worst-case retrieval latency—with a formal safety guarantee (Theorem 1) and six experiments characterising its behaviour, degradation, and real-agent viability. As AI agents move from single-purpose assistants to interconnected participants in enterprise data workflows, the systems that govern what those agents remember will increasingly determine what those agents are permitted to say, and to whom; content-and-role governance answers “who owns this memory,” while derivation-gated governance answers the equally necessary question, “could this requester have produced this result themselves.”

VIII. REPRODUCIBILITY, DATA, AND ETHICS

The reference implementation (Python; AMU/lineage data model, three competing memory systems, event-replay simulator, statistical analysis, and the sqlglot-based real-agent pipeline) reproduces every table and figure in this paper and is containerised via Docker Compose for one-command reproduction; full module-by-module documentation is in the Supplementary Material. No proprietary or personally identifiable data was used: the synthetic and TPC-H-inspired schemas use generated or benchmark data, and the Northwind database (Experiment 6) is a decades-old public fictional dataset containing no real individuals’ information. This work does not involve human subjects or data requiring ethical review. The threat model (§V) is deliberately explicit about what the mechanism does not protect against, because overstating a safety guarantee’s scope is itself a governance risk. Source code, generated datasets, and raw per-seed results will be made available at a public repository upon

acceptance, and are available from the corresponding author upon reasonable request in the interim.

CONFLICT OF INTEREST

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

ACKNOWLEDGMENT

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors. The authors thank no additional contributors beyond those listed in the author list.

References

- [1] Digital Applied, "AI agent adoption 2026: 120+ enterprise data points," Digital Applied, Tech. Rep., 2026, aggregating Gartner and S&P Global Market Intelligence/McKinsey survey data. [Online]. Available: <https://www.digitalapplied.com/blog/ai-agent-adoption-2026-enterprise-data-points>
- [2] IBM Security, "Cost of a data breach report 2025," IBM, Tech. Rep., 2025. [Online]. Available: <https://www.ibm.com/reports/data-breach>
- [3] F. El Yagoubi, G. Badu-Marfo, and R. Al Mallah, "AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems," arXiv preprint arXiv:2602.11510, 2026.
- [4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, 2023.
- [5] P. Rasmussen, "Zep: A temporal knowledge graph architecture for agent memory," arXiv preprint arXiv:2501.13956, 2025.
- [6] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic memory for LLM agents," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2025, arXiv:2502.12110.
- [7] H. Taheri, "Governed memory: A production architecture for multi-agent workflows," arXiv preprint arXiv:2603.17787, 2026.
- [8] C. Lam, J. Li, L. Zhang, and K. Zhao, "Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework," arXiv preprint arXiv:2603.11768, 2026.
- [9] Oracle Corporation, "Oracle AI agent memory: A governed, unified memory core for enterprise AI agents," Oracle Developer Blog, Mar. 2026. [Online]. Available: <https://blogs.oracle.com/developers/oracle-ai-a-gent-memory-a-governed-unified-memory-core-for-enterprise-ai-agents>
- [10] Colrows, "Why BI metrics do not match across dashboards," Colrows Technical Blog, 2024. [Online]. Available: <https://colrows.com/blogs/why-bi-metrics-do-not-match-across-dashboards/>
- [11] Snowflake, Salesforce, dbt Labs, BlackRock, and RelationalAI, "Open Semantic Interchange (OSI): A vendor-neutral semantic layer specification for AI and BI," Industry Standard Initiative, Sep. 2025. [Online]. Available: <https://www.snowflake.com/en/news/press-releases/snowflake-sales-force-dbt-labs-and-more-revolutionize-data-readiness-for-ai-with-open-semantic-interchange-initiative/>
- [12] European Parliament and Council, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (EU AI Act)," Official Journal of the European Union, Jul. 2024. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689>
- [13] P. Du, "Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers," arXiv preprint arXiv:2603.07670, 2026.
- [14] N. Abaev, D. Klimov, G. Levinov, D. Mimran, Y. Elovici, and A. Shabtai, "AgentGuardian: Learning access control policies to govern AI agent behavior," arXiv preprint arXiv:2601.10440, 2026.
- [15] P. Subramaniam and S. Krishnan, "DePLOI: Applying NL2SQL to synthesize and audit database access control," arXiv preprint arXiv:2402.07332, 2024.
- [16] L. Moreau, B. Clifford, J. Freire, J. Futrelle, Y. Gil, P. Groth, N. Kwasnikowska, S. Miles, P. Missier, J. Myers, B. Plale, Y. Simmhan, E. Stephan, and J. Van den Bussche, "The open provenance model—core specification (v1.1)," Future Gener. Comput. Syst., vol. 27, no. 6, pp. 743–756, 2011, doi: 10.1016/j.future.2010.07.005. See also W3C PROV-DM: <https://www.w3.org/TR/prov-dm/>.
- [17] M. Herschel, R. Diestelkämper, and H. Ben Lahmar, "A survey on provenance: What for? What form? What from?," The VLDB J., vol. 26, no. 6, pp. 881–906, 2017, doi: 10.1007/s00778-017-0486-1.
- [18] S. H. Zhang, Z. Miao, and J. Wang, "LINEAGEX: A column lineage extraction system for SQL," in Proc. 41st IEEE Int. Conf. Data Eng. (ICDE), Demo Track, 2025, arXiv:2505.23133.
- [19] A. Naik, J. Culligan, Y. Gal, P. Torr, R. Aljundi, A. Paren, and A. Bibi, "OMNI-LEAK: Orchestrator multi-agent network induced data leakage," arXiv preprint arXiv:2602.13477, 2026.
- [20] M. Alizadeh, Z. Samei, D. Stetsenko, and F. Gilardi, "Simple prompt injection attacks can leak personal data observed by LLM agents during task execution," arXiv preprint arXiv:2506.01055, 2025.
- [21] P. Wang, Y. Li, and Y. Tian, "Aligning provenance with authorization: A dual-graph defense for LLM agents," arXiv preprint arXiv:2605.26497, 2026.
- [22] Transaction Processing Performance Council, TPC-H Benchmark Specification, Revision 3.0.1. TPC, 2021. [Online]. Available: <http://www.tpc.org/tpch/>
- [23] T. Mao and contributors, "sqlglot: SQL parser and transpiler," 2023, MIT Licence, version 23+ used in this work. [Online]. Available: <https://github.com/tobymao/sqlglot>
- [24] Microsoft Corporation, Northwind Traders Sample Database, 2000, canonical fictional business dataset. [Online]. Available (open-source SQLite port): <https://github.com/jpwhite3/northwind-SQLite3>
- [25] H. Chase and contributors, "LangChain: Building applications with LLMs through composability," 2023, MIT Licence. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [26] Ollama contributors, "Ollama: Run LLMs locally," 2024, MIT Licence, local LLM server used in Experiment 6. [Online]. Available: <https://github.com/ollama/ollama>
- [27] A. Rezazadeh, Z. Li, A. Lou, Y. Zhao, W. Wei, and Y. Bao, "Collaborative memory: Multi-user memory sharing in LLM agents with dynamic access control," arXiv preprint arXiv:2505.18279, 2025.
- [28] Z. Ren, Y. Yang, Y. Chen, Z. Zhao, B. Fu, Z. Shu, B. Zhang, Y. Xu, D. Guo, and S. Yan, "GateMem: Benchmarking memory governance in multi-principal shared-memory agents," arXiv preprint arXiv:2606.18829, 2026.

VENKATA SANGARAJU is an independent researcher working on memory and data-governance architectures for enterprise artificial intelligence (AI) agents. His research interests include lineage and provenance tracking for AI agent systems, privacy-preserving retrieval, column-level access control, and the application of formal safety guarantees to shared multi-agent memory infrastructure. He is the corresponding author of this work (ORCID: 0009-0001-7716-1342).

SUDHIR VISSA is with SAGE7 AI, Georgetown, Texas, USA. His research interests include enterprise AI agent architectures, data governance, and applied infrastructure for multi-agent deployments, including the metric-definition and access-control problems addressed in this work (ORCID: 0009-0003-7865-0863).